



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escola Tècnica Superior d'Enginyeria Informàtica
Universitat Politècnica de València

Glosario de Conceptos

Trabajo Fin de Grado

Grado en Informática Industrial y Robótica

Autor: Lourdes Francés Llimerá

Tutor: Juan Francisco Blanes Noguera

2025-2026

Tabla de contenidos

Tabla de contenidos.....	2
Índice de tablas.....	3
1. Contexto	4
2. Términos externos.....	4
3. Términos propios de LARIC.....	7
4. Flujo de extremo a extremo.....	8

Índice de tablas

Tabla 1. Términos Externos.....	7
Tabla 2. Términos Propios.	8

1. Contexto

Este glosario recoge los términos técnicos más relevantes para comprender el sistema LARIC (Language-based Agent for Robotic Interaction and Control). Para cada término se indica el área a la que pertenece y su significado concreto dentro del proyecto. Los términos se agrupan en dos tablas: conceptos externos (tecnologías y estándares de terceros) y conceptos propios del sistema LARIC.

2. Términos externos

Término	Área	Definición
Agente	IA	Entidad de inteligencia artificial capaz de razonar sobre una petición y decidir de forma autónoma qué acción o herramienta invocar para resolverla. En LARIC el agente recibe el mensaje del usuario, lo envía al LLM y ejecuta la herramienta que el modelo selecciona.
Action Server	ROS 2	Mecanismo de comunicación para tareas de larga duración con tres fases: Goal Request (solicitud), Feedback (progreso) y Result (estado terminal). En LARIC se usa para /spin, /drive_on_heading, /backup y /navigate_to_pose.
AMCL (Adaptive Monte Carlo Localization)	Nav2	Algoritmo de localización probabilístico que estima la posición del robot en el mapa cruzando las lecturas del LIDAR con el mapa pregenerado. En LARIC (simulación) requiere inicialización manual con “2D Pose Estimate”.
Behavior Plugin	Nav2	Complementos del stack de Nav2 que exponen action servers para movimientos primitivos seguros: /spin (giro en sitio), /drive_on_heading (avance en línea recta), /backup (retroceso).
Callback	Python	Función registrada para ejecutarse automáticamente al ocurrir un evento específico. En LARIC hay callbacks de ROS 2 (llegada de mensajes a un tópico suscrito) y de Qt (eventos de teclado y audio).
Costmap	Nav2	Mapa de costes que combina el mapa estático y las lecturas del LIDAR para representar zonas libres y ocupadas. Nav2 lo usa para planificar rutas y para la comprobación de colisión de los Behavior Plugins.
Daemon thread	Python	Hilo marcado con “daemon=True” que se termina automáticamente al acabar el proceso principal. En LARIC los hilos “think()” en agent_logic.py son daemons para no bloquear la salida del programa.
DDS (Data Distribution Service)	Middleware	Estándar de conectividad empleado como capa de transporte subyacente en ROS 2. Basado en un modelo de publicación/suscripción, permite comunicación en tiempo real, descubrimiento automático de nodos y configuración de la calidad de servicio (QoS) para asegurar la entrega de

		datos.
Debounce	HMI	Mecanismo que ignora cambios rápidos en una señal durante un tiempo mínimo. En LARIC evita que una pulsación imprecisa la barra espaciadora corte la grabación de voz accidentalmente.
Foxglove	Visualización	Herramienta web de visualización de datos ROS 2. La sirve el snap husarion-webui. En LARIC (real) sustituye a RViz para ver el mapa y publicar la pose inicial.
Gazebo	Simulación	Simulador físico 3D. En LARIC (simulación) carga el mundo turtlebot3_house con el modelo TurtleBot3 Waffle Pi y publica sensores y TF.
geometry_msgs/Twist	ROS 2	Mensaje estándar para comandos de velocidad. Contiene linear.x (m/s), angular.z (rad/s). Se publica en /cmd_vel para mover el robot.
GoalStatus	ROS 2	Código numérico que indica el estado terminal de un Action Server. Los valores relevantes para LARIC son: 4 (STATUS_SUCCEEDED), 5 (STATUS_CANCELED), 6 (STATUS_ABORTED).
Groq	IA	Proveedor de inferencia LLM en la nube con hardware LPU especializado de muy baja latencia. En LARIC sirve el modelo llama-3.3-70b-versatile cuando se trabaja desde fuera del laboratorio.
Husarion ROSbot 3	Hardware	Robot móvil de Husarion empleado como plataforma real. Ejecuta Ubuntu y ROS 2 Jazzy con LIDAR RPLIDAR y cámara DepthAI gestionados mediante snaps.
LangChain	IA	Librería que orquesta el ciclo de razonamiento del agente (patrón ReAct). En LARIC gestiona la construcción del prompt, la llamada al LLM y la ejecución de la herramienta seleccionada.
LIDAR	Sensores	Sensor de distancias por láser. En LARIC, alimenta el costmap de Nav2 para la localización y la evasión de obstáculos.
Llama 3.3 70B	IA	LLM de Meta con 70.000 millones de parámetros. En LARIC actúa como el núcleo de razonamiento del agente que interpreta la intención del usuario y decide qué acción ejecutar.
LLM (Large Language Model)	IA	Modelo de inteligencia artificial entrenado con enormes cantidades de texto que puede comprender y generar lenguaje natural.
msgfmt / gettext	i18n	Sistema estándar de internacionalización. Los textos se extraen a un catálogo .po, se traducen y se compilan a .mo binario que se carga en tiempo de ejecución.
Nav2	ROS 2	Stack de navegación autónoma para ROS 2. En LARIC gestiona toda la capa de movimiento: localización (AMCL), planificación de rutas y

		ejecución de movimientos primitivos vía Behavior Plugins.
Odometría	Sensores	Estimación de posición y orientación a partir de los encoders de las ruedas. Se publica en /odom (nav_msgs/Odometry).
Ollama	IA	Plataforma para servir LLMs vía API REST. En LARIC sirve Llama 3.3 70B en la infraestructura del instituto ai2 de la UPV.
PTT (Push-to-Talk)	HMI	Modalidad de captura de audio activada manualmente. En LARIC se activa mediante la barra espaciadora e incluye debounce de 150 ms y filtro de duración mínima de 0.2 s.
Publicador / Suscriptor	ROS 2	Patrón de comunicación asíncrona de ROS 2: un publisher envía mensajes a un tópico sin saber quién los recibirá y todos los subscribers activos en ese tópico reciben los mensajes automáticamente.
pyqtSignal	PyQT5	Mecanismo de señales de Qt para comunicación segura entre hilos. En LARIC permite que hilos secundarios actualicen el log de la UI sin tocar directamente el widget (evita condiciones de carrera).
RAI (Robotec AI Framework)	Framework	Actúa como puente entre ROS 2 y LLMs. Proporciona ROS2Context, ROS2Connector, y herramientas preimplementadas para Nav2 (NavigateToPoseTool, CancelNavigateToPoseTool, etc.)
ReAct	IA	Paradigma de razonamiento (Reasoning + Acting). El agente alterna entre razonar (qué hacer) y actuar (invocar herramienta). LangChain implementa este patrón al AgentExecutor.
ROS 2	Middleware	Middleware de código abierto para robótica que proporciona un bus de comunicación asíncrona (publisher/subscriber) entre nodos de software heterogéneos, apoyado en DDS. En LARIC, permite interconectar de forma sencilla todos los componentes del sistema.
ROS2Connector	RAI	Objeto central de comunicación de RAI. Métodos principales: register_callback (suscribirse a tópicos), send_message (publicar). También gestiona las acciones vía start_action y terminate_action.
snap	Linux	Paquete autocontenido de Ubuntu. En el ROSbot real, la navegación, la webui (Foxglove), DepthAI y RPLIDAR se distribuyen como snaps.
SLAM Toolbox	Nav2	Paquete de SLAM 2D. En el robot real se usa primero en modo mapeo (SLAM:=True) para crear el mapa y después en modo localización (SLAM:=False) contra el mapa guardado.
STT (Speech-to-Text)	HMI	Tecnología que convierte una señal de audio en texto escrito.

		En LARIC se usa la API de Google Speech Recognition.
System Prompt	IA	Instrucción de texto que define el comportamiento del LLM.
		En LARIC contiene las reglas de selección de herramientas, las convenciones de signo y restricciones de ejecución.
TF (Transform Library)	ROS 2	Árbol de transformadas que relaciona los sistemas de coordenadas del robot (map, odom, base_link, etc.). Su consistencia es clave para la localización y la navegación.
TurtleBot3 Waffle	Hardware	Robot diferencial de Robotis con LIDAR 360° y encoders.
		En LARIC es el modelo simulado en Gazebo.
2D Pose Estimate	Nav2/RViz	Herramienta de Rviz para fijar manualmente la posición inicial del robot. Publica en /initialpose.
		En el robot real la pose se publica de forma equivalente desde Foxglove.

Tabla 1. Términos Externos.

3. Términos propios de LARIC

Término	Área	Definición
abort_event	LARIC	threading.Event compartido. StopTool lo activa (set) y SpinTool, MoveTool y NavigationTool lo consultan (is_set) en sus bucles de polling para cancelar la acción en curso inmediatamente.
motion_lock	LARIC	threading.Lock compartido. Cada herramienta de movimiento intenta adquirirlo de forma no bloqueante al entrar; si está ocupado, rechaza el comando. Garantiza que solo haya un movimiento a la vez. StopTool no lo adquiere, para poder interrumpir siempre.
localised_event	LARIC	threading.Event compartido. Se activa al recibir un mensaje en /initialpose. Todas las herramientas de movimiento lo comprueban como prerequisite.
Parada de emergencia	LARIC	Mecanismo que evita por completo el agente de IA. Dos capas: el botón de la UI (Ctrl+E) publica un Twist de zeros directamente en /cmd_vel, y el tópico /emergency_stop notifica al agente para que ejecute StopTool.
Mapa semántico	LARIC	Diccionario KNOWN_LOCATIONS en config.py que traduce nombres en lenguaje natural a coordenadas del mapa (x, y, yaw). Existen dos conjuntos (simulación y robot real), seleccionados por is_real_robot.
is_from_lab	LARIC	Conmutador de config.py. True = inferencia local con Ollama en el ai2 de la UPV; False = inferencia en la nube con Groq.
is_real_robot	LARIC	Conmutador de config.py que selecciona el conjunto de ubicaciones y el script de apagado adecuado (simulación o robot real).
/from_human	LARIC	Tópico (std_msgs/String) por el que la HMI envía el texto del usuario (voz transcrita o teclado) al agente.
/robot_feedback	LARIC	Tópico (std_msgs/String) por el que el agente envía a la HMI los mensajes de estado que se muestran en el log.
/language_changed	LARIC	Tópico (std_msgs/String) por el que la HMI difunde el código de idioma activo para que el proceso del agente sincronice su catálogo gettext.
return_direct	LARIC	Atributo de las herramientas LangChain. A True, la salida de

la herramienta se devuelve directamente sin pasar de nuevo por el LLM, lo que reduce latencia y consumo de tokens.
--

Tabla 2. Términos Propios.

4. Flujo de extremo a extremo

Resumen del recorrido completo desde que el usuario habla hasta que el robot se mueve:

1. El usuario habla (PTT) o escribe en `laric_interface.py`.
2. Si es voz, Google STT transcribe el audio a texto.
3. La interfaz publica el texto en `/from_human`.
4. El agente (`agent_logic.py`) lo recibe en `on_human_input` y lanza un hilo `think()`.
5. LangChain construye el prompt (`SYSTEM_PROMPT` + mensaje) y llama al LLM.
6. El LLM decide qué herramienta invocar y con qué parámetros.
7. La herramienta comprueba `localised_event` y `motion_lock` y envía el goal al action server de Nav2 correspondiente.
8. El bucle de polling espera el `GoalStatus` (4/5/6) o `abort_event`.
9. El agente publica el resultado en `/robot_feedback`.
10. La interfaz muestra el resultado, con código de colores, en el log.